EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

If a player chooses an occupied square, the program should switch to the other player anyway.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

DO NOW — RECALL

Noughts & Crosses · Part 3 — Make moves

Name: _____ Class: _____ Date: _____

1. Which method from Part 2 displays the board?

2. In a zero-indexed C# 2D array, which row and column does board[1,2] access?

3. What do nested loops let us do with the board?

4. Move 5 is the centre square. What zero-based row and column should it map to?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

DrawBoard().

2. Model answer

Row 1, column 2.

3. Model answer

Traverse every row and column.

4. Model answer

[1,1].

By the end, you can…

- map a move from 1–9 to a row and column;
- write MakeMove() so occupied squares are rejected;
- alternate correctly between X and O after valid moves.

Vocabulary — know these terms

Recall the term — 2D array

Recall the term — Row

Recall the term — Column

Recall the term — Nested loop

Recall the term — Validation

Recall the term — Game loop

Recall the term — Method

Recall the term — State

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Write SwitchPlayer()
Return O when given X, otherwise return X.

Question 4

Make alternating turns work
Use your Part 2 DrawBoard() , prompt for a move, call MakeMove() , and switch player only after a valid move.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Map a move
Write code that converts an integer move from 1–9 into zero-based row and column values.

Question 2

Write MakeMove()
Create MakeMove(int move, string player) . Reject out-of-range and occupied squares; otherwise place the symbol and return true.

NEW LEARNING

Read, annotate and explain

Today: make the board interactive

move 1 → [0,0]
move 5 → [1,1]
move 9 → [2,2]

valid → place X/O → switch player
invalid → reject → same player retries

Checkpoint: by the end, two players can make valid alternating moves on the Part 2 board.

Map 1–9 into row/column

Integer division finds the row; MOD finds the column. The picture highlights move 8: row 2, column 1. Inside MakeMove(), check the 1–9 range before using these indexes.

Check the move before accessing the array

Reject choices outside 1–9 before calculating/using their indexes. Reject occupied squares too. Never overwrite an X or O.

Make the move

A Boolean return value tells Main whether the move succeeded.

Switch player only after success

An invalid move should leave the same player to try again.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Turn move 8 into a board position

Subtract 1

```
move - 1 = 7
```

Board indexes begin at 0.

Integer divide by 3

```
7 / 3 = 2
row = 2
```

Integer division discards the remainder.

Use MOD 3

```
7 % 3 = 1
col = 1
```

The remainder identifies the column.

Place the player

```
board[2,1] = player;
```

Move 8 maps to row 2, column 1.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Map a move

Move 6 is chosen.

```
row = (6 - 1) / 3 = ?
col = (6 - 1) % 3 = ?
```

Method / working:

2. Validate range

Reject choices outside 1–9.

```
if (move < ? || move > ?)
{
    return false;
}
```

Method / working:

3. Reject occupied square

The target may already contain X or O.

```
if (board[row,col] == ? ||
    board[row,col] == ?)
{
    return false;
}
```

Method / working:

4. Switch player

Return the other symbol.

```
if (player == "X")
    return ?;
else
    return ?;
```

Method / working: